

Detailed Walkthrough: Ant Farm Reinforcement Learning System

1. Overview

The Ant Farm system is a **reinforcement learning (RL) environment** where an agent ("ant") learns to navigate a 2D grid toward a **fixed goal** using **Q-learning**. The system saves learned data across runs to improve efficiency.

2. Key Components & Code Structure

The project is modular, consisting of several files:

Main Components

File	Purpose
<code>env.py</code>	Defines the environment where the agent moves and interacts.
<code>q_learning.py</code>	Implements the Q-learning algorithm for training.
<code>main.py</code>	Orchestrates the training and visualization process.
<code>config.py</code>	Stores hyperparameters and environment settings.
<code>utils.py</code>	Handles saving/loading Q-tables and plotting results.

3. How Each Module Works

A. `config.py` – Configuration Settings

This file defines all parameters that control how the system runs.

Key Settings:

- `GRID_SIZE = 10` → The size of the environment grid (10x10).
- `MAX_STEPS = 100` → Limits the number of steps per episode.
- `FIXED_GOAL = (5, 5)` → The goal position is fixed at (5,5).
- `NUM_EPISODES = 5000` → Number of training episodes.
- `Q_TABLE_FILE = "q_table.npy"` → Saves the trained Q-table for persistence.

B. `env.py` – The AntFarm Environment

Defines a **custom Gymnasium environment** where the agent moves inside a 2D grid.

Key Features:

- The agent starts at a random location.
- The goal remains fixed at `(5, 5)`.
- The agent can move **up, down, left, or right**.

The reward is based on **Manhattan distance**:

`reward = 10 - new_distance`

- - Extra **+50 points** when reaching the goal.
- The environment resets when an episode ends.
- Uses **Pygame** to render movement.

C. `q_learning.py` – Q-Learning Algorithm

This module implements the RL learning process.

Algorithm Breakdown:

1. **Initialize** the Q-table (or load from a file).
2. **Loop through episodes**
 - Start from a random state.

Pick an action using an **ϵ -greedy policy**:

`if np.random.random() < epsilon:`

`action = env.action_space.sample() # Explore`

`else:`

```
action = np.argmax(Q[state_key]) # Exploit
```

-
- Take a step in the environment.

Update the Q-value using the Bellman equation:

```
Q[state_key][action] += alpha * (reward + gamma * best_next - Q[state_key][action])
```

- - Reduce **epsilon** (less exploration over time).
3. **Save the Q-table** after training.

D. **utils.py** – Utility Functions

Provides helper functions for:

Saving & loading the Q-table:

```
def save_q_table(Q, file_path):  
    np.save(file_path, Q)
```

-
- **Plotting training results** (reward per episode, steps taken).

E. **main.py** – Running the Simulation

The main script that ties everything together.

Execution Flow:

1. **Initialize the environment** (**AntFarmEnv**).
2. **Load the Q-table** if it exists.
3. **Train the agent using Q-learning**.
4. **Save the updated Q-table**.
5. **Render the trained agent moving in the grid**.

```
if __name__ == '__main__':  
    main()
```

4. How the System Works in Practice

A. Training Phase

- The agent **randomly explores** at first.
- Over time, it **learns the best path** to the goal.
- The Q-table is saved to ensure **knowledge retention** across runs.

B. Running a Trained Model

- If a **Q-table already exists**, training **resumes** instead of starting over.
- The agent **optimally navigates** to the goal using learned data.

C. Visualizing Learning Progress

- The **training process is plotted** to show improvement over episodes.
 - The **agent's movement is displayed** using Pygame.
-

5. Summary

- The system uses **Q-learning** to train an agent to navigate a **grid world**.
- The goal remains **fixed** at (5,5).
- Training results are **saved** so the agent gets better over time.
- A **Pygame visualization** shows the learning process in action.
- The system is **modular**, making it easy to tweak and extend.

Let me know if you need further clarifications! 🚀